

Análisis de Componentes Principales (PCA)

Base de datos Iris — Código desarrollado en Python / Jupyter Notebook

Código

```
# Tratamiento de datos
import numpy as np
import pandas as pd

# Gráficos
import matplotlib.pyplot as plt

# Preprocesado y modelado
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from numpy import linalg as LA

# Cargar la base de datos
df = pd.read_csv("iris.csv")
df.head()

df.shape

# Determinar el número de renglones de la base de datos
index = df.index
renglones = len(index)
renglones

# Adaptación de la base para quitar la variable no numérica
df_pca = df.iloc[:, 0:4]
df_pca

# Estandarización de las variables
df2 = StandardScaler().fit_transform(df_pca)
df2

# Cálculo de la matriz de correlaciones
A = (1 / renglones) * np.dot(df2.T, df2)
A

# Entrenamiento del modelo PCA con el escalado de los datos
pca_pipe = make_pipeline(StandardScaler(), PCA())
pca_pipe.fit(df_pca)

# Se extrae el modelo entrenado del pipeline
modelo_pca = pca_pipe.named_steps["pca"]

# Eigenvalores
print("Eigenvalores:")
eigenvalores = LA.eigvals(A)
print(eigenvalores)

# Porcentaje de la varianza explicada por cada componente
print("Porcentaje de varianza explicada por componente")
print(modelo_pca.explained_variance_ratio_)

# Cálculo de los eigenvectores
print("Eigenvectores (por renglón):")
pd.DataFrame(
    data=modelo_pca.components_,
    columns=df_pca.columns,
    index=["PC1", "PC2", "PC3", "PC4"]
)

# Proyecciones de los componentes
proyecciones = np.dot(modelo_pca.components_, df2.T)
proyecciones = pd.DataFrame(
    proyecciones,
    index=["PC1", "PC2", "PC3", "PC4"]
)
proyecciones = proyecciones.transpose().set_index(df.index)
proyecciones

# Mapa de observaciones
```

```

x = proyecciones.iloc[:, 0]
y = proyecciones.iloc[:, 1]
z = df.index
x = x.to_numpy()
y = y.to_numpy()

fig, ax = plt.subplots()
ax.set_title("Mapa de Observaciones")
ax.scatter(x, y)

for i, txt in enumerate(z):
    ax.annotate(txt, (x[i], y[i]))

plt.show()

# Mapa de factores
componentes_2 = pd.DataFrame(
    data=modelo_pca.components_,
    columns=df_pca.columns,
    index=["PC1", "PC2", "PC3", "PC4"]
)
componentes_2 = componentes_2.iloc[0:2, :]
componentes_2 = componentes_2.T
componentes_2

x = componentes_2.iloc[:, 0]
y = componentes_2.iloc[:, 1]
z = componentes_2.index
x = x.to_numpy()
y = y.to_numpy()

fig, ax = plt.subplots()
ax.set_title("Mapa de Factores")
ax.scatter(x, y)

for i, txt in enumerate(z):
    ax.annotate(txt, (x[i], y[i]))

plt.show()

# Conclusión:
# En el mapa se observa que sepal.length, petal.length y
# petal.width tienen mayor relación con PC1, mientras que
# sepal.width tiene mayor relación con PC2.

# Procedimiento para obtener la matriz estandarizada original
original = np.dot(modelo_pca.components_.T, proyecciones.T)
original = original.T
print(original)

```